1. What are Props?

Props (short for *properties*) are used to **pass data from one component to another** in React.

- Props are **read-only** (cannot be changed by the child).
- They help make components **dynamic and reusable**.
- Data usually flows **from parent to child**.

Example

```
function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}

function App() {
  return <Welcome name="Alex" />;
}
```

Output:
Hello, Alex!

---

2. Passing Data to Components

You pass data to a component using **attributes**, similar to HTML.

Parent Component

```
<Student name="Sam" age={15} />
```

Child Component

```
function Student(props) {
  return (
    <p>
      Name: {props.name}, Age: {props.age}
    </p>
  );
}
```

Using Destructuring (Recommended)

```
function Student({ name, age }) {
  return <p>Name: {name}, Age: {age}</p>;
}
```

✔ Cleaner and easier to read.

---

3. Props vs Variables

Props	Variables
Passed to components	Declared inside components
Read-only	Can be changed
Used for communication	Used for logic/storage
External data	Internal data

Example

```
function Counter({ start }) {
  let count = start; // variable
  return <p>Count: {count}</p>;
}
```

- `start` → **prop**
- `count` → **variable**

---

4. Reusable Components

A **reusable component** is one component used multiple times with different props.

Example: Button Component

```
function Button({ text, color }) {
  return (
```

```
    <button style={{ backgroundColor: color }}>
      {text}
    </button>
  );
}
```

Using the Same Component Multiple Times

```
<Button text="Save" color="green" />
<Button text="Delete" color="red" />
<Button text="Edit" color="blue" />
```

✔ One component, many uses.

---

5. Default Props

Default props are used when no value is passed.

Example

```
function Greeting({ name = "Guest" }) {
  return <h2>Welcome, {name}!</h2>;
}
```

Usage

```
<Greeting />
<Greeting name="Lina" />
```

Output:

```
Welcome, Guest!
Welcome, Lina!
```

□ Practical Task (Try This!)

Task: Create a `ProfileCard` Component

Requirements:

1. Create a component called `ProfileCard`
2. It should accept props:
   - `name`
   - `age`
   - `hobby`
3. Set a **default hobby** as `"Reading"`
4. Display the data neatly
5. Reuse the component **at least 2 times**

1. What is State?

State is a built-in React object used to **store and manage data that can change** over time in a component.

Key Points

- State is **mutable** (can change).
- When state changes, the **component re-renders**.
- State is **local to a component**.

Example

```
function Counter() {
  let count = 0; // normal variable
  return <h1>{count}</h1>;
}
```

 This will NOT update the UI when changed.

```
function Counter() {
  const [count, setCount] = React.useState(0);
  return <h1>{count}</h1>;
}
```

✔ State updates the UI automatically.

---

2. useState Hook

`useState` is a **React Hook** that allows functional components to use state.

Syntax

```
const [stateValue, setStateValue] = useState(initialValue);
```

Example

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+</button>
    </>
  );
}
```

---

3. Handling Events (Click, Input)

React handles events using **camelCase** syntax.

Click Event

```
<button onClick={handleClick}>Click Me</button>
function handleClick() {
  alert("Button clicked!");
}
```

Input Event

```
function NameInput() {
```

```
  const [name, setName] = useState("");

  return (
    <>
      <input
        type="text"
        onChange={(e) => setName(e.target.value)}
      />
      <p>Your name: {name}</p>
    </>
  );
}
```

---

4. Updating State Correctly

□ Wrong Way

```
count = count + 1;
```

React will NOT re-render.

✔ Correct Way

```
setCount(count + 1);
```

✔ Best Practice (Using Previous State)

```
setCount(prevCount => prevCount + 1);
```

✔ Use this when the new state depends on the old state.

---

5. Controlled Components (Forms)

A **controlled component** is a form element whose value is **controlled by React state**.

Example: Controlled Input

```
function LoginForm() {
  const [email, setEmail] = useState("");

  return (
    <form>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <p>Email entered: {email}</p>
    </form>
  );
}
```

✔ React controls the input value.

□ Practical Task (Hands-On)

Task: Create a Simple Form

Requirements:

1. Create a component called `UserForm`
2. Add state for:
   - `username`
   - `age`
3. Use **controlled inputs**
4. Display entered values live
5. Add a button to **reset the form**